INDICODE

A Complete Blueprint for an AI-Native Programming Language

Language design | Compiler front-end | C++ implementation roadmap | AI systems

MASTER DESIGN SPECIFICATION / VERSION 1.0

Research baseline: 2 November 2026

Prepared as a beginner-readable engineering blueprint.

Document status and how to use this blueprint

This is a design and implementation blueprint, not a claim that a production language already exists. It defines a proposed language, a recommended compiler architecture, a 777-item capability catalogue, staged engineering work, and measurable acceptance criteria. The catalogue is a roadmap inventory: the entries are not all implemented, and they are not 777 primitive data types.

Important scope boundary

A language can be specified comprehensively, but no document can guarantee that it will become as fast, mature, stable, or widely adopted as C, C++, or Python. Those outcomes require implementation, testing, users, ecosystem work, and years of measured iteration. This blueprint makes that work concrete without promising impossible results.

Reading path for a beginner

1. Read Parts I-III first to understand the language goal and the 22 concepts to learn.
2. Read Parts IV-VI to see how the syntax, types, AI features, and compiler front-end fit together.
3. Use Part VII as the C++ engineering map. Build one milestone at a time and keep each milestone testable.
4. Use Part VIII and the 777-item catalogue as a long-term capability register, not a checklist to implement all at once.
5. Use the research references to study the established compiler and AI systems that informed the design.

What this blueprint promises - and what it does not

This blueprint provides	This blueprint does not claim
A coherent proposed syntax and semantic rules	An already completed production compiler
An exact 777-entry concept catalogue in 22 families	777 built-in scalar types or 777 implemented features
A beginner-first set of 22 learning foundations	That learning only 22 ideas makes someone automatically an AI scientist
A from-scratch C++ front-end architecture and staged roadmap	That an entire GPU/AI compiler can be built in one step
Research references and design trade-offs	That every research choice is universally best

Executive summary

Indicode is proposed as a readable, statically checked, AI-native programming language with three coordinated programming levels: high-level application programming, systems-level programming, and device-level programming. Its front-end should be written in C++20 or newer, beginning with a custom lexer, parser, abstract syntax tree, symbol resolver, type checker, and diagnostic engine. Later stages lower typed source into an Indicode intermediate representation (IR), perform semantics-preserving optimizations, and connect to CPU, GPU, and portable execution backends.

The central usability idea is progressive disclosure: beginners use a small set of predictable rules; advanced users can ask for exact memory, numeric, concurrency, and device behavior; AI researchers use first-class tensor, differentiation, model, dataset, and training concepts. Compiler defaults should make common programs concise, but every important implicit action should be inspectable and overridable.

Table of contents

- PART I - Mission, design principles, and product boundaries
- PART II - The proposed Indicode language: syntax and semantics
- PART III - The 22 foundations a beginner should learn
- PART IV - AI-native programming model
- PART V - Compiler front-end: complete design
- PART VI - Intermediate representation, optimization, and backends
- PART VII - C++ engineering architecture and build plan

- PART VIII - 777-concept capability catalogue
- PART IX - Roadmap, tests, risks, and acceptance criteria
- PART X - Research references and glossary

PART I - Mission, design principles, and product boundaries

1. The mission

Create a new language named Indicode for people who want to build software and AI systems without first learning a large collection of unrelated tools. The source code should read like straightforward English instructions while retaining formal, compiler-checkable meaning. The language should be useful for small scripts, large applications, numeric computing, model training, systems software, and hardware-specific kernels.

The design should be friendly to learners in India and internationally. That means plain English keywords, consistent spelling, predictable errors, minimal punctuation where safe, and documentation that explains each concept in everyday terms. It does not mean replacing precise programming semantics with ambiguous natural-language guessing.

2. Five non-negotiable design principles

Readable but unambiguous: Use familiar words such as say, if, repeat, function, and train. Do not let the compiler guess intent from vague prose.

Safe defaults, explicit escape hatches: Use inferred types and managed values for ordinary code. Require explicit opt-in for raw pointers, unchecked memory, data races, and hardware-specific operations.

One semantic core, multiple levels: High-level and low-level code must share the same type, function, module, and diagnostic rules. They should not be separate languages glued together.

AI as a first-class domain: Tensor shapes, numeric precision, automatic differentiation, model graphs, datasets, devices, and training steps must be represented in the language and its IR.

Optimization without semantic surprise: The compiler may optimize by default, but it must preserve specified program behavior. Fast-math, quantization, precision relaxation, and approximate transformations require explicit policies.

3. The three programming levels

Level	Who uses it	Examples	Default guarantees
Level A - Everyday / high-level	Beginners, app developers, AI model authors	Inferred variables, functions, lists, tensors, training loops	Static types, clear errors, managed memory, bounds checks
Level B - Systems / low-level	Runtime, compiler, performance engineers	Exact-width integers, structs, explicit layout, pointers in unsafe scope	Precise layout and integer semantics; unsafe boundaries visible
Level C - Device / hardware-level	Kernel and accelerator specialists	CPU vector kernels, GPU kernels, memory spaces, synchronization	Explicit target, device memory, launch and synchronization rules

The levels are not three different syntaxes. A high-level program may call a low-level library; a low-level function may expose a safe high-level wrapper; an AI graph may lower part of its computation to a device kernel. The compiler must retain type and source information across all boundaries.

4. What should be built versus reused

Build Indicode-specific language design and front-end components from scratch so the language has its own grammar, syntax tree, type rules, diagnostics, and semantics. Reuse mature infrastructure where doing so is more responsible than recreating complex target-specific machinery: LLVM for mature code generation, MLIR when multiple IR levels and dialect lowering become necessary, established GPU vendor toolchains for their hardware, and ONNX for model exchange. Reuse does not mean the Indicode language is merely a wrapper; the language and front-end remain its own.

Recommended strategic choice

Create the Indicode lexer, parser, AST, name resolver, type checker, diagnostics, and initial typed IR in C++. For the first native backend, emit LLVM IR or use LLVM APIs rather than writing machine-code encoders for every CPU. Introduce MLIR only when the compiler genuinely needs multi-level tensor and device lowering.

5. Non-goals for version 1

- Do not attempt to implement every one of the 777 catalogue entries before a usable compiler exists.
- Do not make source code depend on English free-form sentence interpretation. Syntax must be grammatical and deterministic.
- Do not silently change numeric precision or quantize a model merely because it appears faster.
- Do not promise equal performance across different CPUs, GPUs, drivers, and memory systems.
- Do not build a new operating system, GPU driver, deep-learning framework, and programming language simultaneously.

PART II - The proposed Indicode language: syntax and semantics

6. Source files and basic writing style

Proposed source extension: .indi. The canonical keyword spelling is lowercase; built-in type names may accept familiar case aliases such as Int and int, but ordinary identifiers are case-sensitive. A source file is UTF-8. Newlines terminate simple statements when the parser can determine that the statement is complete; semicolons may be accepted as optional separators for generated code and compact statements.

```
say("hello!")


A = 22
B = "a"
Message = "hello"


Int age = 22
Int2 score = 30000
```

The display example uses the requested simple form. The compiler should define say as a standard-library function, not a special case in the parser. That makes it testable, replaceable, and usable as an ordinary function.

7. Variables and type inference

A first assignment without an explicit type declares a mutable variable and infers its type from the initializer. The inferred type becomes fixed for that variable binding. Reassignment is allowed only when the new expression has a compatible type; the compiler must not silently change the variable from number to text.

```
A = 22        # inferred Int (default signed 32-bit)
A = 23        # valid: same type
A = "a"       # error: A was already inferred as Int


Letter = "a"    # inferred Char: exactly one Unicode scalar
Word = "hello"  # inferred Text
```

For explicit declarations, the declared type is authoritative. The compiler must reject a value that cannot be converted under the language conversion rules. Implicit narrowing is forbidden. Explicit conversion functions should be visible in code.

```
Int count = 32
Int count2 = "32"      # type error
Int2 small = 30000     # valid
Int2 tooLarge = 40000  # compile-time range error
Int2 negative = -32768 # valid
Int2 below = -32769    # compile-time range error
```

8. Numeric type rules

Indicode type	Width	Meaning	Range / note
Byte / U8	8-bit	Unsigned integer	0 to 255
Int2 / I16	16-bit	Signed integer	-32,768 to 32,767

Indicode type	Width	Meaning	Range / note
UInt2 / U16	16-bit	Unsigned integer	0 to 65,535
Int / I32	32-bit	Signed integer	-2,147,483,648 to 2,147,483,647
UInt / U32	32-bit	Unsigned integer	0 to 4,294,967,295
Long / I64	64-bit	Signed integer	-2^63 to 2^63-1
ULong / U64	64-bit	Unsigned integer	0 to 2^64-1
Half / F16	16-bit float	Low precision numeric	Use only when selected for supported workloads
Float / F32	32-bit float	Default model scalar	IEEE-style target semantics to be specified
Double / F64	64-bit float	Higher precision scalar	Use for precision-sensitive work
Bool	logical	true or false	Not interchangeable with integers
Char	Unicode scalar	One Unicode scalar value	Not a byte and not necessarily one displayed grapheme
Text	UTF-8 text	Sequence of Unicode text	Length and indexing semantics explicitly defined
Void	none	No returned value	Function result absence

Why Int is fixed by Indicode

C++ built-in int has a minimum width and its exact width is implementation-dependent. Indicode should define Int as exactly 32 bits in its language specification, regardless of the host compiler. The C++ implementation must use exact-width support where available or a verified equivalent, and must fail configuration if it cannot meet the promised representation.

9. Overflow and conversion policy

- Compile-time constants: reject a value outside the destination type range with a precise error.
- Runtime checked arithmetic: the default behavior for integer overflow is a defined Overflow error or checked result, never silent wraparound.
- Wrapping arithmetic: provide explicitly named wrapping operators/functions for algorithms that require modular arithmetic.
- Saturating arithmetic: provide explicitly named saturating operations for signal processing and quantized kernels.
- Narrowing conversion: require an explicit conversion such as to_Int2(value, checked: true).
- Division: define divide-by-zero behavior and signed minimum divided by negative one behavior.
- Floating point: define NaN, infinity, signed zero, rounding modes, contraction/FMA policy, and fast-math opt-in.

10. Character and text rule

To honor the example A = "a", Indicode infers Char when a quoted literal contains exactly one Unicode scalar value; a literal with two or more Unicode scalar values becomes Text. The explicit Text constructor forces text even for one scalar. A one-scalar character is not the same thing as one UTF-8 byte. A visually single grapheme such as a composed accented letter or a joined emoji can contain multiple Unicode scalars; for predictable behavior it should infer Text unless explicitly represented as a single scalar.

```
A = "a"              # Char
B = "hello"          # Text
C = Text("a")        # explicitly Text
D = Char.from_code(65) # Char: Unicode scalar U+0041
```

11. Function and control-flow syntax

```
function add(a: Int, b: Int) -> Int {
   return a + b
}

if score >= 50 {
   say("pass")
```

```
} else {
   say("try again")
}

repeat 5 times {
   say("working")
}

for item in items {
   say(item)
}
```

Function declarations require a stable signature after type checking. Return types may be inferred for small local functions in a future version, but explicit public API signatures are recommended. Loops must define iteration order, mutation rules, and overflow behavior for counters.

12. Proposed compact grammar (illustrative EBNF)

This grammar is a starting point for a formal language specification. The final grammar must be versioned and tested against an exhaustive parser corpus. The lexer handles comments, identifiers, literals, and source positions before parsing.

```
program       = { declaration | statement } ;
declaration   = function_decl | type_decl | module_decl ;
function_decl  = "function" identifier "(" [parameters] ")" ["->" type] block ;
parameters    = parameter { "," parameter } ;
parameter     = identifier ":" type ;
statement     = inferred_decl | typed_decl | assignment | if_stmt
              | loop_stmt | return_stmt | expression_stmt | block ;
inferred_decl  = identifier "=" expression ;
typed_decl    = type identifier "=" expression ;
assignment    = lvalue "=" expression ;
if_stmt       = "if" expression block ["else" block] ;
loop_stmt     = "repeat" expression "times" block
              | "for" identifier "in" expression block
              | "while" expression block ;
block         = "{" { statement } "}" ;
expression    = ... ;  (* precedence table is normative *)
```

13. Operator precedence and evaluation

Precedence (high to low)	Operators / family	Rule
1	call, index, member access	Evaluate primary then postfix operations
2	unary + - ! ~	Right-to-left; type checked
3	* / %	Numeric types must be compatible
4	+ -	Numeric add/subtract; text concatenation is a separate operation
5	< <= > >=	Comparable operands only
6	== !=	Equality only for types with defined equality

Precedence (high to low)	Operators / family	Rule
7	&&	Short-circuit logical AND
8	||	Short-circuit logical OR
9	=	Assignment; right side evaluated before write

The final language specification should include a complete precedence table, associativity, evaluation order, side-effect ordering, integer promotion rules, and generic overload resolution. Ambiguities should be errors, not compiler guesses.

14. Type safety, mutability, and scope

- Variables have one static type after declaration or successful inference.
- let introduces an immutable binding; var or inferred assignment introduces a mutable binding. The exact syntax can be finalized during the language RFC.
- Scopes are lexical. A local name is visible from its declaration to the end of its containing scope, subject to nested shadowing policy.
- No implicit conversion may lose information. Numeric widening may be allowed only when defined as value-preserving.
- Null-like absence is represented by Option<T>, not by an untracked null pointer.
- Functions declare effects such as file access, network, device execution, or mutation when the effect system is introduced.

PART III - The 22 foundations a beginner should learn

The following 22 foundations form the beginner curriculum. A learner should be able to build small programs and a first educational AI project using these foundations. Advanced compiler, hardware, and mathematical capabilities can be added later through libraries, runtime defaults, guided tooling, and optional specialist syntax.

#	Foundation	Beginner outcome
1	Values and expressions	A value is data; an expression computes a value.
2	Variables	Give a name to a value and update it when allowed.
3	Type inference	Let the compiler infer a type from the first value.
4	Explicit types	Write a type when exact meaning matters.
5	Basic types	Understand integers, floats, Bool, Char, and Text.
6	Conversions and overflow	Know when a value fits and when conversion is unsafe.
7	Functions	Group reusable steps under a name.
8	Conditions	Choose between paths with if and else.
9	Loops	Repeat work with repeat, while, and for.
10	Arrays and records	Store related values in a predictable shape.
11	Collections	Use lists, maps, sets, and iterators.
12	Modules	Split a project into files and reusable packages.
13	Errors	Return, handle, or propagate a failure explicitly.
14	Mutation and immutability	Understand when data may change.
15	Memory basics	Know value ownership, references, and lifetime at a simple level.
16	Tensor shapes and dtypes	Describe AI arrays by shape and numeric type.
17	Tensor operations	Use elementwise operations, matrix products, and reductions.
18	Model composition	Combine layers and functions into a model.
19	Automatic differentiation	Compute gradients from a differentiable function.
20	Datasets and batching	Load, transform, shuffle, and batch examples.
21	Training and evaluation	Run a training loop and measure model quality.
22	Build, run, test, and debug	Compile a project, find errors, and verify behavior.

Learning outcomes after the 22 foundations

- Write a small command-line program and split it into modules.
- Read a dataset, validate it, and create tensor batches.
- Define a simple model using standard layers and a forward function.
- Train a small model with a supported optimizer and loss function.
- Evaluate and compare model behavior without confusing training accuracy with generalization.
- Use tests and compiler diagnostics to find and fix errors.
- Recognize when low-level control is needed and when a library is safer.

The correct promise

Twenty-two foundations are a sensible starting curriculum, not a guarantee of expertise. They should make a learner productive in beginner projects. Becoming an AI scientist also requires mathematics, experimental design, statistics, domain knowledge, critical reading, and sustained practice.

PART IV - AI-native programming model

15. AI must be part of the language model

AI support should not be limited to calling Python libraries from Indicode. The compiler should understand tensor shape, element type, device placement, differentiability, data layout, and computation graph structure. The runtime may call high-performance libraries, but the compiler must have enough information to validate and optimize programs safely.

16. Tensor type design

```
images: Tensor<Float32, [Batch, 224, 224, 3], Device.Auto>
weights: Tensor<Float16, [768, 3072], Device.GPU>

function forward(x: Tensor<Float32, [Batch, 784]>) -> Tensor<Float32, [Batch, 10]> {
  return x * weights + bias
}
```

The syntax above is illustrative. A tensor type should carry element dtype, rank, dimensions (including symbolic dimensions), layout metadata where needed, and device or placement constraints. Shape checking should catch contradictions before execution when shapes are static, and emit runtime guards when dimensions are dynamic.

17. Essential AI primitives

Area	Required language concepts	Compiler/runtime responsibility
Tensor core	Tensor, Shape, DType, Layout, Device, Buffer	Shape/type inference, broadcasting validation, layout conversions
Numerical ops	MatMul, Conv, Reduce, Normalize, Softmax, Activation	Kernel selection, fusion, precision policy, stable numerical behavior
Autodiff	requires_grad, grad, backward, stop_gradient	Build derivative graph or tape; define supported derivative rules
Model graph	Layer, Parameter, Module, Forward, State	Represent model graph; serialize stable model metadata
Training	Loss, Optimizer, Step, Scheduler, Checkpoint	Validate training loop; update parameters; save/recover state
Data	Dataset, Transform, Batch, Sampler, DataLoader	Efficient streaming, shuffling, caching, reproducibility
Deployment	Export, Quantize, Compile, Serve, Profile	Model export, target-specific compilation, performance reporting

18. Example: readable training workflow

```
data = Dataset.load("train.data")
data = data.shuffle(seed: 42).batch(size: 32)

model = Model {
  Dense(784, 256)
  ReLU()
  Dense(256, 10)
}

optimizer = Adam(model.parameters, rate: 0.001)
```

```
repeat 10 times {
  for batch in data {
    prediction = model.forward(batch.input)
    loss = CrossEntropy(prediction, batch.label)
    optimizer.update(model, loss)
  }
}

say("training complete")
```

This is a proposed surface-level workflow, not an implemented API. The detailed semantics must define train/eval modes, gradient clearing, loss reduction, data shuffling, random seeds, batch remainder handling, checkpoint format, and exception behavior. The compiler should not invent these details.

19. Automatic differentiation design

Start with reverse-mode automatic differentiation for scalar losses over many parameters, because this is the common training pattern. Add forward-mode for workloads with few inputs and many outputs, then compose higher-order differentiation only after the base rules are well tested. Use explicit differentiability rules per operator. Nondifferentiable operations must be documented, rejected in strict gradient regions, or require a clearly named custom derivative rule.

- Every differentiable primitive must have a derivative rule and gradient tests.
- Gradient graph construction must record only operations the compiler/runtime can observe.
- Mutation and hidden side effects inside traced functions need strict rules.
- Checkpointing/recomputation may trade compute for memory, but must preserve the mathematical result within the stated numeric policy.
- A gradient check compares automatic derivatives with finite differences where appropriate; it is a test tool, not a proof.

20. Quantization, low precision, and BitNet-style experimentation

Indicode should expose low-bit and quantized tensor types as explicit experimental numeric formats, not silently convert ordinary Float32 model parameters. A format such as ternary {-1, 0, +1} values has algorithmic and training implications beyond storage width. The type system should represent scale, zero point, packing, accumulator type, and rounding/saturation policy when applicable. Quantization-aware training and post-training quantization should be distinguishable operations.

PART V - Compiler front-end: complete design

21. What the compiler front-end does

The front-end turns source text into a validated, typed representation that later compiler stages can use. It does not need to generate machine instructions. The front-end must report errors precisely and preserve enough source information for editor tools, debugging, formatting, and diagnostics.

22. Front-end pipeline

Stage	Responsibility
1. Source manager	Read UTF-8 source files; track file IDs, byte offsets, line and column positions.
2. Lexer / tokenizer	Convert characters into tokens; handle identifiers, literals, comments, operators, and invalid characters.
3. Parser	Apply the formal grammar; build AST nodes; recover from common syntax errors.
4. AST / syntax model	Represent declarations, statements, expressions, types, and source spans.
5. Name resolution	Resolve variables, functions, types, modules, imports, and overload candidates.
6. Type checking / inference	Check assignments, calls, operators, conversions, generic constraints, and tensor shapes.
7. Semantic validation	Check return paths, mutability, visibility, effects, ownership and unsafe boundaries.
8. Typed HIR	Produce a semantically valid high-level IR with explicit inferred types and conversions.
9. Diagnostics and tooling	Emit errors, warnings, notes, fix suggestions, symbol indexes, and language-server data.

23. Source manager and lexer

The source manager owns immutable source buffers and maps byte offsets to line/column positions. It must use UTF-8 validation and preserve original source spelling. The lexer emits tokens containing kind, source span, and optional literal payload. It should be deterministic, fast, and independent of semantic meaning.

- Token families: identifiers, keywords, integer/float literals, strings, punctuation, operators, comments, newline separators, end-of-file, invalid token.
- Numeric literal parsing must detect malformed digits, separators, exponents, suffixes, and overflow before semantic typing.
- String parsing must define escapes, raw strings, Unicode escapes, invalid UTF-8, and source encoding rules.
- Lexer errors must be recoverable so one malformed token does not hide every later error.
- Keep comments and trivia available for formatters and documentation tools even if the parser discards them.

24. Parser strategy

Recommended first parser: recursive descent for declarations and statements, plus a Pratt parser for expressions. This keeps the initial implementation understandable and makes operator precedence explicit. Grammar changes require parser tests and an update to the language specification. Parser functions should return structured success/error results rather than terminating the process.

25. Abstract syntax tree design

Use tagged node kinds with compact source spans. Store nodes in an arena or another lifetime-safe owner to avoid thousands of tiny allocations. AST nodes should represent syntax rather than guessed machine behavior. Keep a separate typed representation so type checking does not mutate syntax nodes unpredictably.

AST family	Representative nodes
Declarations	ModuleDecl, FunctionDecl, VariableDecl, TypeDecl, ImportDecl
Statements	BlockStmt, IfStmt, WhileStmt, ForStmt, ReturnStmt, ExpressionStmt
Expressions	NameExpr, LiteralExpr, CallExpr, BinaryExpr, UnaryExpr, IndexExpr, MemberExpr
Types	NamedType, GenericType, FunctionType, TensorType, ReferenceType
Metadata	SourceSpan, Attribute, Visibility, DocumentationComment

26. Name resolution and symbol tables

Use nested lexical scopes with symbol tables. Each scope tracks variables, functions, types, modules, and imported names in separate namespaces if the language semantics require it. Resolve a name once and attach a stable symbol identity to the typed representation. Diagnostics should distinguish an unknown name, an inaccessible private name, an ambiguous import, and an overload mismatch.

27. Type checking and inference

Type inference should be local and predictable for version 1. Start with literal typing, declaration inference, function argument checks, return type checks, operator typing, and explicit conversions. Avoid global whole-program inference until there is a concrete reason to accept its compile-time and diagnostic complexity.

- Literal rules: integer literals choose the smallest permitted default integer type according to a documented rule; plain 22 is Indicode Int unless out of range, then the compiler asks for an explicit wider type rather than guessing silently.
- Assignment: the left binding type never changes after its declaration.
- Function calls: argument count, named arguments, generic constraints, and overloads are checked before IR lowering.
- Control flow: all reachable return paths must satisfy the declared return type.
- Numeric operations: mixed signed/unsigned and integer/float operations require explicit, documented conversion rules.
- AI tensor types: verify rank, dimensions, broadcast behavior, dtype, and device compatibility when statically knowable.

28. Error recovery and diagnostics

An excellent compiler is judged partly by the quality of its errors. Keep parsing after a recoverable error, use expected-token sets, synchronize at statement boundaries, and produce multiple useful errors without cascading nonsense. Preserve source ranges on all AST and typed IR nodes.

```
example.indi:4:15: error[E-TYPE-014]: cannot assign Text to Int
 4 | Int age = "twenty"
   |           ^^^^^^^^
 note: Int is a 32-bit signed integer.
 help: use Text age = "twenty" or parse an integer explicitly.
```

Diagnostics must have stable error codes, severity, primary span, secondary spans, explanatory text, and optional machine-applicable fixes. A --json-diagnostics mode should support editors and CI systems.

29. Front-end test strategy

Test group	Minimum coverage
Lexer	Every token, malformed literal, UTF-8 edge case, comment, line ending
Parser	Valid grammar examples, precedence, nesting, missing delimiter, recovery

Test group	Minimum coverage
AST	Node shape, source spans, serialization/debug printer
Resolver	Nested scopes, shadowing, imports, visibility, overload candidates
Type checker	Valid/invalid assignments, conversion, range checks, return types, generic errors
Tensor semantics	Static shape agreement, broadcasting, dtype, device mismatch
Diagnostics	Golden tests for error code, location, message, note, fix-it
Robustness	Fuzz lexer/parser; random malformed input must not crash

PART VI - Intermediate representation, optimization, and backends

30. Why Indicode needs more than one IR level

A single representation is unlikely to serve ordinary application code, tensor graphs, low-level memory operations, and target-specific kernels equally well. Use a staged lowering model. Keep source-level information until diagnostics/debugging no longer need it, then progressively make control flow, types, memory, and hardware behavior explicit.

31. Recommended IR layers

Layer	Role	Typical content
AST	Faithful syntax representation	Source structure, names as written, literal spelling
Typed HIR	High-level typed program	Resolved symbols, inferred types, explicit conversions, effects
MIR / Control-flow IR	Lowered language semantics	Basic blocks, branches, explicit temporaries, SSA values
Tensor / Graph IR	AI computation representation	Tensor operators, shapes, layouts, device placement, differentiation metadata
Kernel / Loop IR	Hardware-near computation	Loops, tiles, vector lanes, buffers, synchronization
Target IR	Backend-specific representation	LLVM IR, GPU-specific IR, WebAssembly or other supported target

32. SSA, basic blocks, and data flow

The MIR should use Static Single Assignment (SSA) form or a clearly specified equivalent. Each SSA value is assigned once; control-flow merges use phi-like block arguments. SSA makes data dependencies explicit and supports constant propagation, dead-code elimination, common-subexpression elimination, and many other analyses. The exact IR must be versioned and have a verifier.

33. Optimization strategy

- Correctness first: every optimization pass must preserve defined semantics or be guarded by an explicit approximation policy.
- Pass pipeline: canonicalization, constant folding, dead-code elimination, simplification, inlining, loop transforms, vectorization, memory planning, target lowering.
- Pass manager: declare pass inputs/outputs, dependencies, analyses preserved, and verification points.
- Diagnostics: allow users to print IR before/after a pass and explain why a transformation did or did not apply.
- Reproducibility: record compiler version, target features, optimization profile, random seeds, and auto-tuning choices.
- Compile-time budget: cap expensive search and auto-tuning by default; let users configure it.

34. AI and tensor lowering

Tensor graph lowering must handle operator fusion, layout choice, shape specialization, memory planning, precision conversion, and device scheduling. The compiler should distinguish algebraic equivalence from approximate equivalence. For example, quantizing Float32 weights to a low-bit representation changes numeric behavior and therefore must be explicitly requested and measured.

35. Backends and portability

Backend	Role in roadmap	Caution
Interpreter / bytecode	Fast feedback for early language versions	Not the final high-performance path
LLVM CPU backend	Native code for supported host architectures	Target ABI, debug info, optimization versioning matter
GPU backend	Accelerator kernel code and launch support	Depends on vendor toolchains, hardware and drivers
WebAssembly	Portable sandboxed deployment option	Not a replacement for every native/GPU workload
ONNX export/import	Model interoperability	Operator versions, shape semantics and unsupported ops need validation
External kernel libraries	High-performance math and tensor operations	ABI, licensing, packaging, and reproducibility must be managed

PART VII - C++ engineering architecture and build plan

36. Implementation language and baseline

Implement the compiler front-end in modern C++. Recommended baseline: C++20, with C++23 considered only when compiler support and build environments are stable. Use CMake for portable project configuration and CTest or an equivalent harness for automated tests. Keep external dependencies small in the first milestones.

37. Proposed repository layout

```
indicode/
  docs/                language specification, RFCs, tutorials
  include/indicode/    public C++ headers
  src/
    driver/            command-line interface and compiler options
    source/            source manager, UTF-8 and source locations
    lexer/             tokens and tokenizer
    parser/            parser and AST construction
    ast/               node definitions and AST utilities
    sema/              name resolution, type checking, validation
    diagnostics/       error codes, messages, fix-it support
    ir/                typed HIR and MIR definitions
    passes/            canonicalization and optimization passes
    runtime/           core runtime and standard library hooks
    backends/          interpreter, LLVM, device adapters
    ai/                tensor, graph, autodiff and training interfaces
  tests/
    lexer/ parser/ sema/ diagnostics/ ir/ ai/ integration/ fuzz/
  examples/            beginner programs and AI examples
  tools/               formatter, language server, package tools
  CMakeLists.txt
  CONTRIBUTING.md
  LICENSE
```

38. Core C++ modules and responsibilities

Module	Responsibility	Design constraint
SourceManager	Own source buffers and source spans	No dangling string views after compilation unit ends
Lexer	Produce token stream	No semantic symbol lookup in lexer
Parser	Build AST	No hidden code generation in parser
ASTContext	Own syntax nodes	Clear ownership and stable node identities
Resolver	Bind identifiers to symbols	Separate declaration collection from use resolution if needed
TypeChecker	Infer/check types and conversions	One authoritative type system
DiagnosticEngine	Collect and render diagnostics	Stable codes; no printing directly from parser
IRBuilder	Build typed HIR/MIR	Verify each IR unit
PassManager	Run transformations	Pass contract, test and verify
CompilerDriver	Coordinate compilation	Keep CLI policy out of core semantic libraries

39. First compiler commands

Command	Purpose	Expected output
indicode check file.indi	Lex, parse, resolve, type-check	Success or structured errors
indicode tokens file.indi	Display token stream	Token kind, spelling, source position
indicode ast file.indi	Display parsed AST	Tree view with source spans
indicode typed file.indi	Display typed HIR	Inferred types and resolved symbols
indicode fmt file.indi	Format source	Stable formatted source
indicode build file.indi	Compile program	Object or executable when backend exists
indicode run file.indi	Run through interpreter/runtime	Program output and runtime errors
indicode test	Run test suite	Pass/fail summary and test names

40. Development workflow

6. Write a short language design note before implementing a new feature.
7. Add positive examples and negative examples to the specification.
8. Add lexer/parser/type-checker tests before or alongside the implementation.
9. Update diagnostics and formatter behavior when syntax changes.
10. Run the full test suite and sanitizer builds before merging.
11. Record performance baselines; do not accept speed claims without repeatable benchmarks.
12. Keep a changelog and versioned language compatibility policy.

41. Suggested build dependencies

Keep the first release intentionally small: C++ standard library, CMake, a test framework, and optionally a Unicode library when full Unicode normalization/grapheme operations are required. Later add LLVM libraries for native code generation, MLIR for multi-level IR if justified, and accelerator SDKs only for supported targets. Each dependency needs a version policy, build instructions, license review, security updates, and supported-platform matrix.

PART VIII - The 777-concept capability catalogue

The catalogue contains exactly 777 named design concepts across 22 families. They are deliberately broader than data types: a programming language is made of syntax, types, semantics, compiler stages, runtime services, AI operations, tooling, interoperability, and deployment. The catalogue is a product backlog and language-design index. It does not imply that all entries belong in the beginner syntax or must be implemented in v1.

How to read the catalogue

Core means a capability needed for a reliable small language or a usable AI workflow. Advanced means it belongs after the core compiler exists. Research means it requires benchmarking, formal semantics, or hardware/domain-specific validation before standardization.

Concept family	Entries	Priority
Lexical structure and source syntax	30	Core
Primitive values and data types	36	Core
Type system and inference	40	Core
Variables, constants, scopes, and ownership	36	Core
Operators and expressions	36	Core
Control flow and patterns	30	Core
Functions, generics, and abstractions	36	Core
Records, collections, and data modelling	36	Core
Modules, packages, and APIs	28	Core
Errors, effects, and contracts	30	Core
Concurrency and parallel programming	36	Advanced
Memory, systems, and unsafe control	40	Advanced
Compile-time programming and metaprogramming	30	Advanced
Front-end, semantic analysis, and diagnostics	40	Core
Intermediate representation and optimization	40	Advanced
Tensor and numerical computing	40	Advanced
Automatic differentiation and training	40	Advanced
Model graphs and AI architecture	34	Advanced
Datasets and data pipelines	34	Advanced
Hardware, devices, and target backends	38	Advanced
Quality, tooling, security, and observability	38	Advanced
Runtime, interoperability, packaging, and deployment	29	Advanced
TOTAL	777	22 families

01. Lexical structure and source syntax

Planned entries: 30 | Priority: Core. How source text becomes tokens and language grammar.

ID	Concept
001	UTF-8 source files
002	Unicode identifier policy
003	Identifier start rules
004	Identifier continuation rules
005	Keyword registry
006	Reserved word policy
007	Case sensitivity
008	Whitespace handling
009	Newline statement rules
010	Optional semicolon rules
011	Line comments
012	Block comments
013	Nested comment policy
014	Integer literal forms
015	Binary integer literals
016	Octal integer literals
017	Hexadecimal integer literals
018	Digit separators

ID	Concept
019	Decimal floating literals
020	Exponent notation
021	Numeric suffixes
022	String literal forms
023	Raw string literals
024	Escape sequences
025	Unicode escapes
026	Character literal inference
027	Text literal inference
028	Punctuation tokens
029	Operator tokenization
030	End-of-file and invalid-token rules

02. Primitive values and data types

Planned entries: 36 | Priority: Core. Exact meaning and representation of common values.

ID	Concept
031	Bool
032	Byte
033	U8
034	I8
035	Int2
036	UInt2
037	I16
038	U16
039	Int
040	UInt
041	I32
042	U32
043	Long
044	ULong
045	I64
046	U64
047	I128
048	U128
049	F16
050	BF16
051	F32
052	F64
053	F128 research type
054	Char
055	Text
056	Void
057	Never
058	Unit
059	Decimal fixed-point
060	Duration
061	Timestamp
062	UUID value
063	Option value
064	Result value
065	Complex number
066	Rational number

03. Type system and inference

Planned entries: 40 | Priority: Core. Rules that make values compatible or incompatible.

ID	Concept
067	Static typing
068	Type inference

ID	Concept
069	Local inference
070	Explicit annotation
071	Type identity
072	Type aliases
073	Nominal types
074	Structural types
075	Generic types
076	Generic constraints
077	Trait or interface bounds
078	Union types
079	Option types
080	Result types
081	Nullable value policy
082	Reference types
083	Mutable reference types
084	Immutable reference types
085	Array types
086	Slice types
087	Tuple types
088	Function types
089	Closure types
090	Tensor types
091	Shape types
092	Device-qualified types
093	Effect-qualified function types
094	Type unification
095	Type substitution
096	Overload resolution
097	Coercion rules
098	Explicit conversion
099	Numeric promotion
100	Subtyping policy
101	Variance rules
102	Const qualification
103	Type inference diagnostics
104	Type display formatting
105	Type serialization
106	Type compatibility versioning

04. Variables, constants, scopes, and ownership

Planned entries: 36 | Priority: Core. How values are named, updated, scoped, and kept alive.

ID	Concept
107	Variable declaration
108	Inferred binding
109	Typed binding
110	Immutable let binding
111	Mutable var binding
112	Constant declaration
113	Global variable
114	Local variable
115	Parameter binding
116	Shadowing policy
117	Lexical scope
118	Block scope
119	Module scope
120	Visibility rules
121	Initialization rules
122	Definite assignment
123	Reassignment rules
124	Destructuring binding
125	Tuple binding

ID	Concept
126	Pattern binding
127	Move semantics
128	Borrow semantics
129	Shared borrow
130	Mutable borrow
131	Lifetime region
132	Lifetime inference
133	Drop and cleanup
134	Resource ownership
135	Reference counting option
136	Arena lifetime
137	Thread-local binding
138	Static storage
139	Capture by value
140	Capture by reference
141	Binding diagnostics
142	Ownership visualization

05. Operators and expressions

Planned entries: 36 | Priority: Core. How calculations and value-producing expressions behave.

ID	Concept
143	Arithmetic addition
144	Arithmetic subtraction
145	Multiplication
146	Division
147	Remainder
148	Unary plus
149	Unary minus
150	Logical NOT
151	Bitwise NOT
152	Bitwise AND
153	Bitwise OR
154	Bitwise XOR
155	Left shift
156	Right shift
157	Equality
158	Inequality
159	Ordering comparisons
160	Logical AND
161	Logical OR
162	Short-circuit semantics
163	Assignment
164	Compound assignment
165	Range expression
166	Membership test
167	Index expression
168	Slice expression
169	Member access
170	Function call expression
171	Conditional expression
172	Null coalescing
173	Pipeline operator research
174	Text interpolation
175	Text concatenation
176	Checked arithmetic
177	Wrapping arithmetic
178	Saturating arithmetic

06. Control flow and patterns

Planned entries: 30 | Priority: Core. How a program makes choices and repeats operations.

ID	Concept
179	If statement
180	Else branch
181	Else-if branch
182	While loop
183	Repeat-count loop
184	For-each loop
185	Range loop
186	Break statement
187	Continue statement
188	Return statement
189	Match expression
190	Pattern matching
191	Wildcard pattern
192	Literal pattern
193	Tuple pattern
194	Record pattern
195	Enum variant pattern
196	Guard clause
197	Early return
198	Loop labels
199	Labeled break
200	Loop else research
201	Exhaustiveness checking
202	Unreachable branch check
203	Tail-call position
204	Short-circuit control flow
205	Exception boundary
206	Async control flow
207	State machine lowering
208	Control-flow graph

07. Functions, generics, and abstractions

Planned entries: 36 | Priority: Core. Reusable code, parameters, and abstraction contracts.

ID	Concept
209	Function declaration
210	Function call
211	Parameter types
212	Return types
213	Named arguments
214	Default arguments
215	Variadic arguments
216	Function overloading
217	Generic functions
218	Generic type parameters
219	Generic constraints
220	Trait definitions
221	Trait implementation
222	Interface dispatch
223	Method syntax
224	Associated functions
225	Static methods
226	Extension methods
227	Closures
228	Lambda expressions
229	Closure capture rules
230	Higher-order functions
231	Function pointers

ID	Concept
232	Callback types
233	Pure function marker
234	Effect annotation
235	Inline annotation
236	Calling convention
237	Recursion
238	Tail recursion
239	Async function
240	Generator function
241	Iterator protocol
242	Operator overloading
243	Custom conversion
244	Function documentation

08. Records, collections, and data modelling

Planned entries: 36 | Priority: Core. Ways to represent structured application data.

ID	Concept
245	Struct declaration
246	Enum declaration
247	Tagged union
248	Tuple value
249	Fixed-size array
250	Dynamic array
251	Slice
252	List collection
253	Stack collection
254	Queue collection
255	Deque collection
256	Map collection
257	Hash map
258	Ordered map
259	Set collection
260	Ordered set
261	String builder
262	Bit set
263	Sparse collection
264	Optional field
265	Record update syntax
266	Field visibility
267	Default field value
268	Data class research
269	Iterator protocol
270	Range collection
271	Matrix collection
272	Sparse matrix
273	Graph structure
274	Tree structure
275	Serialization schema
276	Schema validation
277	Pattern destructuring
278	Collection literal
279	Collection comprehension
280	Collection equality

09. Modules, packages, and APIs

Planned entries: 28 | Priority: Core. How projects are split, shared, versioned, and built.

ID	Concept
281	Source module

ID	Concept
282	Module declaration
283	Import declaration
284	Selective import
285	Alias import
286	Public visibility
287	Private visibility
288	Package manifest
289	Package identity
290	Semantic versioning
291	Dependency constraints
292	Lock file
293	Workspace
294	Library target
295	Executable target
296	Build profile
297	Conditional compilation
298	Feature flags
299	Platform configuration
300	Module initialization
301	Circular dependency rules
302	API stability policy
303	Documentation comments
304	Generated documentation
305	Package registry interface
306	Offline dependency cache
307	Reproducible package build
308	License metadata

10. Errors, effects, and contracts

Planned entries: 30 | Priority: Core. How failure and side effects are represented safely.

ID	Concept
309	Compile-time error
310	Runtime error
311	Diagnostic severity
312	Stable error code
313	Source-span diagnostic
314	Related diagnostic note
315	Fix-it suggestion
316	Result type
317	Option type
318	Error propagation
319	Try expression
320	Error mapping
321	Error context
322	Panic policy
323	Assertion
324	Precondition
325	Postcondition
326	Invariant
327	Effect declaration
328	File effect
329	Network effect
330	Device effect
331	Mutation effect
332	Exception interop
333	Cancellation
334	Timeout
335	Retry policy
336	Resource cleanup
337	Error serialization
338	Error recovery semantics

11. Concurrency and parallel programming

Planned entries: 36 | Priority: Advanced. Safe parallel work across tasks, threads, and devices.

ID	Concept
339	Thread abstraction
340	Task abstraction
341	Async/await
342	Future type
343	Promise type
344	Executor
345	Thread pool
346	Work stealing
347	Structured concurrency
348	Task cancellation
349	Task joining
350	Mutex
351	Read-write lock
352	Semaphore
353	Condition variable
354	Atomic value
355	Memory ordering
356	Channel
357	Message passing
358	Actor model
359	Send capability
360	Sync capability
361	Data-race checking
362	Parallel for
363	Parallel reduction
364	Task graph
365	Barrier
366	Latch
367	Event
368	GPU stream abstraction
369	Async device copy
370	Collective communication
371	Distributed process
372	Sharding
373	Backpressure
374	Concurrency diagnostics

12. Memory, systems, and unsafe control

Planned entries: 40 | Priority: Advanced. Precise storage, layout, pointer, and ABI behavior.

ID	Concept
375	Stack allocation
376	Heap allocation
377	Global storage
378	Static storage
379	Arena allocation
380	Pool allocation
381	Manual allocator
382	Allocator interface
383	Raw pointer
384	Non-null pointer
385	Const pointer
386	Mutable pointer
387	Pointer arithmetic unsafe
388	Pointer provenance
389	Reference

ID	Concept
390	Slice reference
391	Bounds checked indexing
392	Unchecked indexing unsafe
393	Memory alignment
394	Struct layout
395	Packed layout
396	Padding inspection
397	Endianness
398	Volatile access
399	Atomic memory access
400	Memory map interface
401	Mapped file
402	Foreign function ABI
403	Calling convention ABI
404	Bit field layout
405	Union layout
406	Placement construction
407	Object lifetime
408	Deterministic destruction
409	Resource handle
410	Memory leak diagnostics
411	Use-after-free checks
412	Aliasing rules
413	Pinning
414	Unsafe block

13. Compile-time programming and metaprogramming

Planned entries: 30 | Priority: Advanced. Generate and validate code before runtime.

ID	Concept
415	Compile-time constant
416	Constant evaluation
417	Compile-time function
418	Static assertion
419	Generic instantiation
420	Template specialization
421	Type-level integer
422	Type-level list
423	Type-level shape
424	Macro declaration
425	Hygienic macro
426	Procedural macro
427	Syntax tree macro API
428	Derive generation
429	Code generation hook
430	Compile-time reflection
431	Field reflection
432	Function reflection
433	Attribute metadata
434	Conditional compilation
435	Target feature query
436	Build script
437	Compile-time resource embedding
438	Schema-to-code generation
439	Parser plugin boundary
440	Macro expansion diagnostics
441	Expansion backtrace
442	Compile-time budget
443	Deterministic macro execution
444	Macro security sandbox

14. Front-end, semantic analysis, and diagnostics

Planned entries: 40 | Priority: Core. The core compiler front-end machinery and its quality.

ID	Concept
445	Source manager
446	Source buffer
447	Source span
448	Line table
449	Column mapping
450	UTF-8 validator
451	Token kind
452	Token payload
453	Lexer state
454	Trivia preservation
455	Parser state
456	Recursive descent parser
457	Pratt expression parser
458	Grammar version
459	AST context
460	AST node arena
461	AST visitor
462	AST printer
463	AST serializer
464	Syntax recovery
465	Error synchronization
466	Symbol identity
467	Scope table
468	Module resolver
469	Import resolver
470	Overload candidate set
471	Name binding
472	Type checker
473	Constraint solver
474	Type substitution engine
475	Constant evaluator
476	Control-flow validation
477	Definite return analysis
478	Effect checker
479	Ownership checker
480	Tensor shape checker
481	Diagnostic engine
482	JSON diagnostics
483	Language server index
484	Incremental parse support

15. Intermediate representation and optimization

Planned entries: 40 | Priority: Advanced. How valid programs are represented and transformed efficiently.

ID	Concept
485	Typed high-level IR
486	Control-flow IR
487	Static single assignment
488	Basic block
489	Block argument
490	Phi merge concept
491	IR verifier
492	IR printer
493	IR serializer
494	IR versioning
495	Dominance analysis
496	Use-def chain
497	Liveness analysis

ID	Concept
498	Alias analysis
499	Escape analysis
500	Constant folding
501	Constant propagation
502	Dead-code elimination
503	Dead-store elimination
504	Common subexpression elimination
505	Canonicalization
506	Inlining
507	Devirtualization
508	Loop invariant code motion
509	Loop unrolling
510	Loop fusion
511	Loop tiling
512	Vectorization
513	Instruction selection
514	Register allocation interface
515	Memory planning
516	Pass manager
517	Pass dependency graph
518	Pass instrumentation
519	Optimization remarks
520	Profile-guided optimization
521	Link-time optimization
522	Incremental compilation
523	Compile cache
524	Debug information

16. Tensor and numerical computing

Planned entries: 40 | Priority: Advanced. Typed tensor computation and numerical behavior.

ID	Concept
525	Tensor value
526	Tensor rank
527	Tensor shape
528	Symbolic dimension
529	Static dimension
530	Dynamic dimension
531	Tensor dtype
532	Tensor layout
533	Tensor stride
534	Tensor view
535	Tensor slice
536	Tensor reshape
537	Tensor transpose
538	Broadcasting
539	Elementwise map
540	Elementwise zip
541	Reduction sum
542	Reduction max
543	Matrix multiplication
544	Batched matrix multiplication
545	Convolution
546	Pooling
547	Normalization
548	Softmax
549	Activation function
550	Gather
551	Scatter
552	Concatenate
553	Stack
554	Split

ID	Concept
555	Padding
556	Mask tensor
557	Sparse tensor
558	Ragged tensor
559	Quantized tensor
560	Complex tensor
561	Random tensor
562	Numerical tolerance
563	Deterministic reduction
564	Tensor shape inference

17. Automatic differentiation and training

Planned entries: 40 | Priority: Advanced. Differentiate computations and train model parameters.

ID	Concept
565	Gradient marker
566	Reverse-mode autodiff
567	Forward-mode autodiff
568	Gradient tape
569	Static derivative graph
570	Dynamic derivative tape
571	Derivative rule registry
572	Custom gradient
573	Higher-order gradient
574	Vector-Jacobian product
575	Jacobian-vector product
576	Jacobian
577	Hessian
578	Stop gradient
579	Gradient accumulation
580	Gradient clipping
581	Gradient checkpointing
582	Loss function
583	Optimizer interface
584	SGD optimizer
585	Momentum optimizer
586	Adam optimizer
587	Learning-rate schedule
588	Weight decay
589	Parameter group
590	Training step
591	Evaluation step
592	Train/eval mode
593	Mixed precision training
594	Loss scaling
595	Gradient overflow check
596	Distributed gradient
597	All-reduce
598	Checkpoint save
599	Checkpoint restore
600	Early stopping
601	Metric tracking
602	Reproducible seed
603	Finite-difference gradient check
604	Training trace

18. Model graphs and AI architecture

Planned entries: 34 | Priority: Advanced. Describe neural networks and other machine-learning models.

ID	Concept
605	Model declaration
606	Model module
607	Layer abstraction
608	Parameter abstraction
609	Buffer state
610	Forward function
611	Sequential model
612	Residual connection
613	Attention operator
614	Multi-head attention
615	Embedding layer
616	Dense layer
617	Convolution layer
618	Recurrent layer
619	Normalization layer
620	Dropout layer
621	Activation catalogue
622	Encoder block
623	Decoder block
624	Transformer block
625	State-space block
626	Mixture-of-experts block
627	Routing gate
628	Model composition
629	Weight tying
630	Shared parameter
631	Model summary
632	Model graph capture
633	Graph validation
634	Model serialization
635	Model import
636	Model export
637	Operator registry
638	Model version metadata

19. Datasets and data pipelines

Planned entries: 34 | Priority: Advanced. Load, validate, transform, and feed data reproducibly.

ID	Concept
639	Dataset interface
640	Streaming dataset
641	In-memory dataset
642	File dataset
643	Columnar dataset
644	Record schema
645	Feature schema
646	Label schema
647	Data validation
648	Missing value policy
649	Data transform
650	Transform composition
651	Tokenizer interface
652	Text normalization
653	Image preprocessing
654	Audio preprocessing
655	Train validation split
656	Test split
657	Shuffle sampler
658	Sequential sampler
659	Weighted sampler
660	Batch sampler
661	Batch padding

ID	Concept
662	Collation function
663	Data loader
664	Prefetch buffer
665	Cache policy
666	Parallel map
667	Parallel filter
668	Data provenance
669	Dataset fingerprint
670	Data versioning
671	Data leakage checks
672	Pipeline profiling

20. Hardware, devices, and target backends

Planned entries: 38 | Priority: Advanced. Map programs onto CPUs, GPUs, and portable targets.

ID	Concept
673	Target triple
674	CPU target
675	GPU target
676	Accelerator target
677	Device type
678	Device placement
679	Host memory
680	Device memory
681	Unified memory hint
682	Pinned host buffer
683	Memory transfer
684	Kernel declaration
685	Kernel launch
686	Thread/block mapping
687	SIMT execution
688	Vector lane
689	SIMD operation
690	Warp or subgroup
691	Shared memory
692	Local memory
693	Global memory
694	Memory coalescing
695	Tensor core capability
696	Atomic device operation
697	Device synchronization
698	Stream scheduling
699	Kernel fusion
700	Kernel specialization
701	Target feature detection
702	CPU vector ISA
703	GPU vendor adapter
704	LLVM backend
705	WebAssembly backend
706	Cross compilation
707	Runtime device discovery
708	Device capability query
709	Kernel profiling
710	Fallback target

21. Quality, tooling, security, and observability

Planned entries: 38 | Priority: Advanced. Make language projects reliable, maintainable, and inspectable.

ID	Concept
711	Unit test

ID	Concept
712	Integration test
713	Golden test
714	Property test
715	Fuzz test
716	Differential test
717	Benchmark test
718	Performance regression test
719	Coverage report
720	Formatter
721	Linter
722	Static analyzer
723	Language server
724	Code completion
725	Go to definition
726	Find references
727	Rename symbol
728	Debugger integration
729	Source map
730	Profiler integration
731	Trace event
732	Structured logging
733	Compiler crash reproducer
734	Sanitizer build
735	Dependency audit
736	Supply-chain metadata
737	Sandbox policy
738	Capability permissions
739	Unsafe audit
740	Secret handling
741	Input validation
742	Resource limit
743	Stack trace
744	Crash report
745	Build reproducibility
746	Hermetic build
747	Accessibility of diagnostics
748	Documentation testing

22. Runtime, interoperability, packaging, and deployment

Planned entries: 29 | Priority: Advanced. Run programs, connect systems, package artifacts, and deploy models.

ID	Concept
749	Runtime library
750	Program entry point
751	Process arguments
752	Environment access
753	Standard input
754	Standard output
755	Standard error
756	File system API
757	Time API
758	Random API
759	Networking interface
760	Foreign function interface
761	C ABI bridge
762	C++ interoperability
763	Python interoperability
764	ONNX interchange
765	Plugin interface
766	Dynamic library
767	Static library
768	Bytecode format

ID	Concept
769	Bytecode verifier
770	JIT compilation
771	AOT compilation
772	Executable packaging
773	Runtime configuration
774	Application bundle
775	Model bundle
776	Serving endpoint
777	Deployment manifest

PART IX - Roadmap, tests, risks, and acceptance criteria

42. The 100-percent roadmap: staged completion, not a calendar promise

The roadmap is organized by dependency and evidence. A milestone is complete only when its acceptance tests pass and the behavior is documented. It is intentionally not assigned a fixed completion date because a first-time language project has uncertain scope and depends on contributor skill, supported hardware, and external toolchain compatibility.

Milestone	Work	Exit criteria
Milestone 0 - Charter and language RFC	Freeze goals, syntax principles, naming, numeric model, string/character policy, error policy, versioning, and non-goals.	Published language charter; open issue list; examples and counterexamples approved.
Milestone 1 - Lexer and source model	Implement UTF-8 source buffers, source spans, tokens, literal parsing, comments, and CLI token output.	Lexer unit tests; malformed-input tests; source locations verified.
Milestone 2 - Parser and AST	Implement declarations, expressions, statements, precedence, AST printer, and recoverable syntax errors.	Parser tests; grammar coverage report; AST dump snapshots.
Milestone 3 - Names and types	Add scopes, symbol identities, inferred variables, explicit types, conversions, integer ranges, and diagnostics.	All examples in Parts II and III pass/fail as specified; no implicit narrowing.
Milestone 4 - Functions and core language	Add functions, records, collections, modules, control flow, errors, and formatter.	Build a multi-file command-line project; integration tests pass.
Milestone 5 - Typed IR and interpreter	Lower validated programs into typed HIR and an interpreter or bytecode.	Run beginner programs without a native backend; IR verifier passes.
Milestone 6 - Native CPU backend	Connect LLVM backend; add debug info, linking, optimization profiles, and target selection.	Native builds pass test suite; benchmarks compared with interpreter.
Milestone 7 - Systems features	Add layout, ownership, references, allocator APIs, FFI, and explicit unsafe blocks.	Memory safety tests; sanitizer CI; ABI compatibility tests.
Milestone 8 - Tensor foundation	Add Tensor types, CPU tensor runtime, shape checking, numeric kernels, and dataset API.	Tensor tests for shape, dtype, layout, broadcasting, and reproducibility.
Milestone 9 - Autodiff and training	Add reverse-mode autodiff, derivative rules, loss, optimizers, checkpoints, and training utilities.	Gradient checks; tiny model training benchmark; save/restore reproducibility.
Milestone 10 - Accelerator execution	Add device abstraction, one GPU backend, kernel execution, transfer management, and profiling.	Correctness against CPU reference; memory and synchronization tests.
Milestone 11 - Ecosystem and release	Stabilize package manager, docs, language server, formatter, model interchange, security process, and compatibility.	Reproducible release build; migration policy; support matrix; public conformance suite.

43. Version strategy

Release	Scope	What users can rely on
0.1	Lexer, parser, AST, basic type checking	Syntax exploration; not stable source compatibility
0.2	Functions, control flow, records, diagnostics	Small typed programs compile/check
0.3	Typed IR, interpreter, standard library core	Runnable portable subset
0.5	Native CPU backend and project modules	Useful small applications
0.7	Systems layer, memory and FFI	Systems programming preview
0.8	Tensor and CPU AI runtime	Educational/small AI workflows
0.9	Autodiff, training, one accelerator backend	AI-native preview
1.0	Stable core syntax, types, packages, conformance suite	Compatibility promise for core language

44. Performance and quality measurements

- Compile time: cold and incremental build duration for fixed source corpora.
- Runtime speed: compare representative programs to equivalent C++ and other baselines under documented optimization and runtime settings.
- Memory use: compiler peak memory, executable memory, tensor peak memory, and allocator overhead.
- AI correctness: operator numerical tolerances, gradient checks, model output agreement, and deterministic mode tests.

- AI performance: training throughput, inference latency, device utilization, memory bandwidth, and end-to-end data loading cost.
- Diagnostics: percentage of negative tests with correct primary location and stable error code.
- Reliability: crash-free fuzzing runs, sanitizer results, reproducible bug reports, and CI pass rate.

45. Security and safety requirements

- Raw pointer and unchecked memory operations require an explicit unsafe scope.
- Package dependencies need pinned versions, integrity hashes, and license/security metadata.
- Compiler and package manager must not execute arbitrary build scripts without explicit project policy.
- Tensor/model loading must validate file sizes, shapes, offsets, and resource limits.
- Device kernels must have clear memory ownership and synchronization rules.
- The compiler must protect against malformed source inputs, stack exhaustion, huge compile-time expansions, and adversarial package metadata.

46. Key risks and how to reduce them

Risk	Why it matters	Mitigation
Scope explosion	Language, compiler, AI runtime, GPU, and tools can overwhelm one person.	Ship a narrow vertical slice; keep 777 items as staged catalogue.
Ambiguous syntax	English-like syntax can become hard to parse.	Use concise fixed grammar and reject ambiguous forms.
Overcomplicated type system	777 visible features would defeat beginner learning.	Expose 22 foundations; hide advanced details behind defaults and libraries.
Numeric surprises	Overflow and conversions can corrupt results.	Specify exact widths, checked defaults, explicit wrapping/saturation.
Backend dependence	Target code generation is large and hardware-specific.	Build own front-end; integrate mature backend infrastructure.
AI performance gap	A tensor syntax alone does not create fast kernels.	Use established kernels, measure, then optimize high-impact operations.
Unstable language changes	Early syntax changes can break examples and packages.	Version the grammar, publish RFCs, keep conformance tests and migration notes.

47. Definition of a successful Indicode 1.0

- The language specification fully defines core lexical, syntax, type, conversion, overflow, memory, and error behavior.
- The C++ compiler front-end passes lexer, parser, semantic, diagnostic, fuzz, and regression test suites.
- The compiler can check and execute documented core programs using a stable runtime.
- At least one native CPU target is supported with debug information and optimization profiles.
- The AI subset includes tensors, shape checks, differentiation, datasets, and a reproducible training workflow.
- A documented path exists for one supported accelerator backend, with explicit platform limits.
- Documentation, formatter, package workflow, test runner, and beginner tutorials are published.
- A conformance suite and compatibility policy define what 1.0 means.

PART X - Research references and glossary

48. Research basis and design mapping

The design borrows established compiler and AI-system patterns rather than assuming a new language should reinvent every layer. The sources below were consulted for architectural guidance. They are references, not endorsements of every feature or a claim that Indicode has implemented their technology.

R1. Clang Internals Manual. Front-end separation, parser/Sema interaction, AST design, source locations, recovery and diagnostics. Official documentation URL: https://clang.llvm.org/docs/InternalsManual.html

R2. LLVM Language Reference Manual. Typed SSA-oriented IR, target triple, data layout, functions and backend representation. Official documentation URL: https://www.llvm.org/docs/LangRef.html

R3. MLIR Dialect Conversion. Multi-level lowering, conversion targets, rewrite patterns, type conversion, partial and full conversions. Official documentation URL: https://mlir.llvm.org/docs/DialectConversion/

R4. ONNX Intermediate Representation Specification. Typed tensors, graph representation, operator sets, data types, model interchange. Official documentation URL: https://onnx.ai/onnx/repo-docs/IR.html

R5. JAX Transformations: grad and vmap. Automatic differentiation, vectorization, function transformations and composition. Official documentation URL: https://docs.jax.dev/en/latest/101/transformations.html

R6. JAX Tracing. Tracing model, abstract values, staged computation, side-effect constraints. Official documentation URL: https://docs.jax.dev/en/latest/tracing.html

R7. PyTorch torch.compiler. Graph capture and compiler stack concepts for deep-learning program acceleration. Official documentation URL: https://docs.pytorch.org/docs/main/user_guide/torch_compiler/torch.compiler.html

R8. Apache TVM Design and Architecture. End-to-end tensor compiler flow, IR transformation, target translation, tensor program optimization. Official documentation URL: https://tvm.apache.org/docs/arch/

R9. Apache TVM TensorIR. Tensor program abstractions, loop transformations, scheduling and tensor-specific optimization. Official documentation URL: https://tvm.apache.org/docs/deep_dive/tensor_ir/index.html

R10. C++ Working Draft N4988. Portable C++ integer-type guarantees and why a source language should define its own fixed-width semantics. Official documentation URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/n4988.pdf

R11. CUDA Programming Guide - Programming Model. Host/device memory, device execution and hardware memory hierarchy. Official documentation URL: https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/programming-model.html

R12. OpenMP Specifications. Portable parallel programming model and API concepts. Official documentation URL: https://www.openmp.org/specifications/

R13. WebAssembly Core Specification. Validated, portable, sandbox-oriented low-level execution model. Official documentation URL: https://www.w3.org/TR/wasm-core/

R14. CMake Tutorial. Portable project configuration, targets, and build-system organization. Official documentation URL: https://cmake.org/cmake/help/latest/guide/tutorial/

Research access date: 2 November 2026. Standards and vendor documentation evolve; the implementation team should re-check the current version before committing to a backend, API, or ABI.

49. Glossary

Term	Meaning
AST	Abstract Syntax Tree: a structured representation of source syntax.
AOT	Ahead-of-time compilation: compile before program execution.
Autodiff	Automatic differentiation: mechanically computes derivatives of program operations.
Backend	Compiler stage that emits code for a target architecture or runtime.
Borrowing	Temporary access to a value without taking ownership.
DType	Element data type used by a tensor.
Effect	A declared kind of observable action, such as I/O or mutation.
HIR	High-level intermediate representation.

Term	Meaning
IR	Intermediate representation: compiler-internal program form.
JIT	Just-in-time compilation: compile during program execution.
MIR	Mid-level intermediate representation, often with explicit control flow.
SSA	Static Single Assignment: each IR value is assigned once.
Tensor	A multi-dimensional array with shape and element type.
Shape	The sizes of a tensor along its dimensions.
Lowering	Transforming a higher-level representation into a more explicit lower-level form.
Quantization	Representing numeric values using a reduced or otherwise changed precision format.
MLIR	Multi-Level Intermediate Representation compiler infrastructure.
ONNX	Open Neural Network Exchange model/intermediate-representation ecosystem.
LLVM	Compiler infrastructure including IR, optimizers and code generation.
Unicode scalar	A Unicode code point excluding surrogate code points.
Grapheme cluster	A user-perceived text character that may contain multiple Unicode scalars.
ABI	Application Binary Interface: conventions for binary-level interoperability.

50. Final design recommendation

Start with the smallest reliable Indicode vertical slice: a source file containing say("hello!"), inferred variables, explicit Int/Int2 declarations, arithmetic, if/else, loops, and functions. Implement the full front-end pipeline in C++ and make every rule testable. Then add modules and a typed IR. Only after programs are reliable should the project add native code generation, systems controls, tensors, autodiff, datasets, training, and accelerator execution.

The defining strength of Indicode should be coherence: one readable language, one type model, one compiler diagnostic system, and a set of progressively lower representations that serve applications, systems code, and AI workloads. The 22 foundations make the language approachable; the 777-item catalogue gives the engineering team a disciplined map for growth.

The next practical action

Treat this document as the project charter. Before coding a feature, write its syntax, semantic rules, positive examples, negative examples, diagnostic requirements, and acceptance tests. Build one verified milestone at a time; do not let the size of the 777-item catalogue delay the first working compiler.